> *prepare a complete guide, similar to VS Code, for Wokwi web, ending with addressing the "On the free plan, Wokwi provides full access to the Public Gateway. The Public Gateway allows the virtual ESP32 to make outbound internet connections (HTTP/HTTPS GET, POST, APIs, MQTT), but blocks direct incoming connections from your local browser." and 'Workaround 1: In-Simulation Self-Test Client (100% Free inside Wokwi)" to correctly run the project completely on cloud.*

> *Just for reference, I am providing the 2 files that were successfully run in VS Code.*

Complete Guide: ESP32 Cloud NAS & Web Node on Wokwi Web

This guide covers setting up, structuring, and running your ESP32 NAS and Static Web Server entirely inside the browser on Wokwi Web, using your glassmorphic UI, dynamic file seeding, and streaming file operations.

Step 1: Create the Project & Circuit Schematics

1. Go to wokwi.com and create a new **ESP32** project.
2. Click on the `diagram.json` tab in the editor.
3. Replace its contents with the following schematic configuration to instantiate the classic ESP32 DevKit and connect it to the MicroSD module over hardware VSPI (`GPIO 5, 18, 19, 23`):

```
{
  "version": 1,
  "author": "Gaus Saraf Murady",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    { "type": "wokwi-microsd-card", "id": "sd1", "top": 0, "left": 220, "attrs": {} }
  ],
  "connections": [
    [ "esp:TX0", "$serialMonitor:RX", "", [] ],
    [ "esp:RX0", "$serialMonitor:TX", "", [] ],
    [ "esp:3V3", "sd1:VCC", "red", [ "v0" ] ],
    [ "esp:GND.1", "sd1:GND", "black", [ "v0" ] ],
    [ "esp:5", "sd1:CS", "yellow", [ "v0" ] ],
    [ "esp:23", "sd1:DI", "blue", [ "v0" ] ],
    [ "esp:19", "sd1:DO", "green", [ "v0" ] ],
    [ "esp:18", "sd1:CLK", "orange", [ "v0" ] ]
  ]
}
```

Step 2: Manage Libraries & Tabs

1. `libraries.txt`: Open this tab and ensure it is **completely empty**. All required drivers (`WiFi`, `WebServer`, `HTTPClient`, `SPI`, `SD`, `FS`) are built directly into the ESP32 Arduino Core.

2. **Create `html_templates.h**`:

- Click the downward arrow (`▼`) or the `+` button near the file tabs in Wokwi.
- Name the new file **`html_templates.h`**.
- Paste your modern glassmorphic UI code into this file:

```
#ifndef HTML_TEMPLATES_H
#define HTML_TEMPLATES_H

#include <Arduino.h>

// ================================================================================
// Modern Glassmorphic Dark UI for ESP32 NAS Dashboard
// ================================================================================
const char NAS_PAGE_HEADER[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ESP32 Cloud NAS & Web Node</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap" rel="styleshee
t">
  <style>
    :root {
      --bg-gradient: radial-gradient(circle at top left, #1a1c29, #0e1017 80%);
      --card-bg: rgba(26, 29, 45, 0.7);
      --card-border: rgba(255, 255, 255, 0.08);
      --accent-cyan: #00d2ff;
      --accent-blue: #3a7bd5;
      --accent-gradient: linear-gradient(135deg, #00d2ff 0%, #3a7bd5 100%);
      --danger-gradient: linear-gradient(135deg, #ff416c 0%, #ff4b2b 100%);
      --success-color: #00e676;
      --text-main: #f0f4f8;
      --text-muted: #94a3b8;
    }
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: 'Inter', -apple-system, sans-serif;
      background: var(--bg-gradient);
      color: var(--text-main);
      min-height: 100vh;
      padding: 30px 20px;
    }
    .container {
      max-width: 900px;
      margin: 0 auto;
```

```
}
.header-card {
  background: var(--card-bg);
  backdrop-filter: blur(12px);
  border: 1px solid var(--card-border);
  border-radius: 16px;
  padding: 24px;
  margin-bottom: 24px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.4);
}
.title-group h1 {
  font-size: 1.5rem;
  font-weight: 700;
  background: var(--accent-gradient);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  margin-bottom: 4px;
}
.title-group p {
  color: var(--text-muted);
  font-size: 0.85rem;
}
.badge-online {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  background: rgba(0, 230, 118, 0.12);
  color: var(--success-color);
  border: 1px solid rgba(0, 230, 118, 0.3);
  padding: 6px 12px;
  border-radius: 20px;
  font-size: 0.8rem;
  font-weight: 600;
}
.badge-dot {
  width: 8px;
  height: 8px;
  background: var(--success-color);
  border-radius: 50%;
  box-shadow: 0 0 8px var(--success-color);
}
.btn-site {
  text-decoration: none;
  background: var(--card-bg);
  border: 1px solid var(--card-border);
  color: var(--accent-cyan);
  padding: 8px 16px;
  border-radius: 8px;
  font-size: 0.85rem;
  font-weight: 500;
  transition: all 0.2s;
}
.btn-site:hover {
  background: rgba(0, 210, 255, 0.1);
```

```
  border-color: var(--accent-cyan);
}
.meter-card {
  background: var(--card-bg);
  backdrop-filter: blur(12px);
  border: 1px solid var(--card-border);
  border-radius: 16px;
  padding: 20px 24px;
  margin-bottom: 24px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.3);
}
.meter-header {
  display: flex;
  justify-content: space-between;
  margin-bottom: 10px;
  font-size: 0.9rem;
}
.progress-bar-bg {
  background: rgba(255, 255, 255, 0.08);
  height: 12px;
  border-radius: 6px;
  overflow: hidden;
  position: relative;
}
.progress-bar-fill {
  height: 100%;
  background: var(--accent-gradient);
  border-radius: 6px;
  transition: width 0.4s ease;
}
.upload-box {
  background: var(--card-bg);
  backdrop-filter: blur(12px);
  border: 2px dashed rgba(0, 210, 255, 0.3);
  border-radius: 16px;
  padding: 24px;
  margin-bottom: 24px;
  text-align: center;
  transition: all 0.2s ease;
}
.upload-box:hover {
  border-color: var(--accent-cyan);
  background: rgba(0, 210, 255, 0.03);
}
.upload-form {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 12px;
  align-items: center;
}
input[type="file"] {
  color: var(--text-muted);
  font-size: 0.85rem;
}
input[type="file"]::file-selector-button {
  background: rgba(255, 255, 255, 0.06);
```

```
  border: 1px solid var(--card-border);
  color: var(--text-main);
  padding: 8px 16px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: 500;
  transition: all 0.2s;
}
input[type="file"]::file-selector-button:hover {
  background: rgba(255, 255, 255, 0.12);
}
.btn-upload {
  background: var(--accent-gradient);
  color: #fff;
  border: none;
  padding: 9px 20px;
  border-radius: 8px;
  font-weight: 600;
  cursor: pointer;
  font-size: 0.85rem;
  transition: transform 0.1s, opacity 0.2s;
}
.btn-upload:hover { opacity: 0.9; }
.btn-upload:active { transform: scale(0.98); }
.table-card {
  background: var(--card-bg);
  backdrop-filter: blur(12px);
  border: 1px solid var(--card-border);
  border-radius: 16px;
  overflow: hidden;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
}
table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
}
th {
  background: rgba(255, 255, 255, 0.03);
  padding: 14px 18px;
  font-size: 0.8rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--text-muted);
  border-bottom: 1px solid var(--card-border);
}
td {
  padding: 14px 18px;
  font-size: 0.9rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.04);
}
tr:last-child td { border-bottom: none; }
tr:hover td { background: rgba(255, 255, 255, 0.02); }
.btn-action {
  display: inline-block;
  text-decoration: none;
```

```
      padding: 6px 14px;
      border-radius: 6px;
      font-size: 0.8rem;
      font-weight: 500;
      margin-right: 6px;
      transition: all 0.2s;
    }
    .btn-dl {
      background: rgba(0, 210, 255, 0.1);
      color: var(--accent-cyan);
      border: 1px solid rgba(0, 210, 255, 0.2);
    }
    .btn-dl:hover {
      background: var(--accent-gradient);
      color: #fff;
    }
    .btn-del {
      background: rgba(255, 75, 43, 0.1);
      color: #ff5252;
      border: 1px solid rgba(255, 75, 43, 0.2);
    }
    .btn-del:hover {
      background: var(--danger-gradient);
      color: #fff;
    }
  </style>
</head>
<body>
  <div class="container">
    <div class="header-card">
      <div class="title-group">
        <h1>ESP32 Storage NAS Node</h1>
        <p>Hardware SPI FAT32 MicroSD Storage Engine</p>
      </div>
      <div style="display: flex; gap: 12px; align-items: center;">
        <span class="badge-online"><span class="badge-dot"></span> Online</span>
        <a href="/site" class="btn-site">🌐 Static Site</a>
      </div>
    </div>
)rawliteral";

const char NAS_PAGE_FOOTER[] PROGMEM = R"rawliteral(
  </div>
</body>
</html>
)rawliteral";

// ============================================================================
// Modern Responsive Landing Page for Static Web Host (/www/index.html)
// ============================================================================
const char SEED_INDEX_HTML[] PROGMEM = R"rawliteral(<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ESP32 Static Edge Host</title>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700&display=swap" rel="stylesheet">
```

```
<style>
  * { margin:0; padding:0; box-sizing:border-box; }
  body {
    font-family: 'Inter', sans-serif;
    background: radial-gradient(circle at 10% 20%, #10121a, #08090d 90%);
    color: #e2e8f0;
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 24px;
  }
  .card {
    max-width: 680px;
    width: 100%;
    background: rgba(20, 24, 38, 0.85);
    backdrop-filter: blur(16px);
    border: 1px solid rgba(255,255,255,0.08);
    border-radius: 20px;
    padding: 40px;
    text-align: center;
    box-shadow: 0 20px 50px rgba(0,0,0,0.5);
  }
  .badge {
    display: inline-block;
    padding: 6px 14px;
    background: rgba(0, 210, 255, 0.12);
    border: 1px solid rgba(0, 210, 255, 0.3);
    color: #00d2ff;
    border-radius: 20px;
    font-size: 0.8rem;
    font-weight: 600;
    margin-bottom: 20px;
  }
  h1 {
    font-size: 2rem;
    font-weight: 700;
    background: linear-gradient(135deg, #00d2ff, #3a7bd5);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    margin-bottom: 12px;
  }
  p.lead {
    color: #94a3b8;
    font-size: 1rem;
    margin-bottom: 30px;
    line-height: 1.6;
  }
  .specs-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
    gap: 16px;
    margin-bottom: 32px;
    text-align: left;
  }
  .spec-item {
    background: rgba(255,255,255,0.03);
```

```
      border: 1px solid rgba(255,255,255,0.06);
      border-radius: 12px;
      padding: 16px;
    }
    .spec-label {
      font-size: 0.75rem;
      color: #64748b;
      text-transform: uppercase;
      letter-spacing: 0.05em;
      margin-bottom: 4px;
    }
    .spec-val {
      font-size: 0.95rem;
      font-weight: 600;
      color: #f1f5f9;
    }
    .btn {
      display: inline-block;
      background: linear-gradient(135deg, #00d2ff, #3a7bd5);
      color: #fff;
      text-decoration: none;
      padding: 12px 28px;
      border-radius: 10px;
      font-weight: 600;
      font-size: 0.9rem;
      transition: transform 0.1s, opacity 0.2s;
    }
    .btn:hover { opacity: 0.9; }
    .btn:active { transform: scale(0.98); }
  </style>
</head>
<body>
  <div class="card">
    <span class="badge">ESP32 Static Edge Host</span>
    <h1>Hosted on MicroSD Storage</h1>
    <p class="lead">
      This static web document is stored as raw blocks on the virtual FAT32 MicroSD card and streamed directly to
your browser over hardware SPI.
    </p>
    <div class="specs-grid">
      <div class="spec-item">
        <div class="spec-label">Microcontroller</div>
        <div class="spec-val">ESP32 DevKit V1</div>
      </div>
      <div class="spec-item">
        <div class="spec-label">Storage Bus</div>
        <div class="spec-val">Hardware VSPI</div>
      </div>
      <div class="spec-item">
        <div class="spec-label">File Protocol</div>
        <div class="spec-val">FAT32 Block I/O</div>
      </div>
      <div class="spec-item">
        <div class="spec-label">Web Engine</div>
        <div class="spec-val">ESP32 WebServer:80</div>
      </div>
    </div>
```

```
    <a href="/" class="btn">📂 Go to NAS File Browser</a>
  </div>
</body>
</html>)rawliteral";

#endif // HTML_TEMPLATES_H
```

Step 3: Implement Main Firmware (`sketch.ino`)

Open the default `sketch.ino` tab, clear any existing code, and paste the complete application:

```
#include <Arduino.h>
#include <WiFi.h>
#include <WebServer.h>
#include <HTTPClient.h>
#include <SPI.h>
#include <SD.h>
#include "html_templates.h"

// Hardware SPI Chip Select pin mapped to GPIO 5
#define SD_CS_PIN 5

// Wokwi virtual Wi-Fi access point credentials
const char* ssid = "Wokwi-GUEST";
const char* password = "";

WebServer server(80);
File uploadFile;

// --- Unit Formatting Helper ---
String formatBytes(size_t bytes) {
  if (bytes < 1024) return String(bytes) + " B";
  if (bytes < (1024 * 1024)) return String(bytes / 1024.0, 1) + " KB";
  return String(bytes / (1024.0 * 1024.0), 1) + " MB";
}

// ==========================================================================
// PHASE 1: STORAGE INITIALIZATION & DIRECTORY SEEDING
// ==========================================================================
void phase1_seedInitialFiles() {
  if (!SD.exists("/www")) {
    SD.mkdir("/www");
  }

  // Seed modern static website to SD card if missing
  if (!SD.exists("/www/index.html")) {
    File html = SD.open("/www/index.html", FILE_WRITE);
    if (html) {
      html.print(FPSTR(SEED_INDEX_HTML));
      html.close();
      Serial.println("[STORAGE] Seeded modern /www/index.html to SD");
    }
  }
```

```
  // Seed default lab report file
  if (!SD.exists("/lab_report.txt")) {
    File report = SD.open("/lab_report.txt", FILE_WRITE);
    if (report) {
      report.println("Course: CSE0611324 Microprocessor & Microcontrollers Lab");
      report.println("Project: ESP32 Embedded NAS & Static Web Node");
      report.println("Architecture: Hardware VSPI (CS:5, MOSI:23, MISO:19, SCK:18)");
      report.println("Filesystem: FAT32 Block Device Driver");
      report.println("Features: File Upload, File Streaming, Delete, Static Web Hosting");
      report.close();
      Serial.println("[STORAGE] Seeded /lab_report.txt to SD");
    }
  }
}

bool phase1_initStorage() {
  Serial.println("\n------------------------------------------------");
  Serial.println("  [PHASE 1 EXECUTION] Mounting MicroSD via VSPI   ");
  Serial.println("------------------------------------------------");

  // Explicitly initialize hardware VSPI bus
  SPI.begin(18, 19, 23, SD_CS_PIN);

  if (!SD.begin(SD_CS_PIN, SPI)) {
    Serial.println("[PHASE 1 ERROR] SD Card initialization failed! Check SPI pins.");
    return false;
  }

  uint64_t totalBytes = SD.totalBytes();
  uint64_t usedBytes = SD.usedBytes();
  Serial.printf("[PHASE 1 SUCCESS] FAT32 Storage Mounted.\n");
  Serial.printf("[PHASE 1 CAPACITY] Total: %llu MB | Used: %llu MB\n",
                totalBytes / (1024 * 1024), usedBytes / (1024 * 1024));

  phase1_seedInitialFiles();

  Serial.println("[PHASE 1 DIRECTORY TREE]:");
  File root = SD.open("/");
  File file = root.openNextFile();
  while (file) {
    Serial.printf("  |- %s (%s)\n", file.name(), formatBytes(file.size()).c_str());
    file = root.openNextFile();
  }
  root.close();

  Serial.println("------------------------------------------------\n");
  return true;
}

// ============================================================================
// PHASE 2: HTTP ROUTE HANDLERS
// ============================================================================

// Route: NAS Dashboard (Renders storage meter, upload form, and file table)
void handleRoot() {
  uint64_t total = SD.totalBytes();
```

```
  uint64_t used = SD.usedBytes();
  float usedPercent = (total > 0) ? ((float)used / (float)total) * 100.0f : 0.0f;

  // Stream header from flash PROGMEM
  server.setContentLength(CONTENT_LENGTH_UNKNOWN);
  server.send(200, "text/html", "");
  server.sendContent(FPSTR(NAS_PAGE_HEADER));

  // Storage meter section
  String meter = "<div class='meter-card'>";
  meter += "<div class='meter-header'>";
  meter += "<span><strong>Storage:</strong> " + formatBytes(used) + " / " + formatBytes(total) + "</span>";
  meter += "<span><strong>" + String(usedPercent, 1) + "%</strong> Used</span>";
  meter += "</div>";
  meter += "<div class='progress-bar-bg'><div class='progress-bar-fill' style='width: " + String(usedPercent, 1) +
"%;'></div></div>";
  meter += "</div>";
  server.sendContent(meter);

  // Upload Form section
  String uploadBox = "<div class='upload-box'>";
  uploadBox += "<form class='upload-form' method='POST' action='/upload' enctype='multipart/form-data'>";
  uploadBox += "<input type='file' name='upload_file' required>";
  uploadBox += "<button type='submit' class='btn-upload'>⬆ Upload to NAS</button>";
  uploadBox += "</form></div>";
  server.sendContent(uploadBox);

  // File Table section
  String tableStart = "<div class='table-card'><table>";
  tableStart += "<tr><th>File Name</th><th>Size</th><th style='text-align: right;'>Actions</th></tr>";
  server.sendContent(tableStart);

  File root = SD.open("/");
  File file = root.openNextFile();
  int fileCount = 0;
  while (file) {
    if (!file.isDirectory()) {
      fileCount++;
      String fname = String(file.name());
      if (fname.startsWith("/")) fname = fname.substring(1);

      String row = "<tr>";
      row += "<td><strong>📄 " + fname + "</strong></td>";
      row += "<td>" + formatBytes(file.size()) + "</td>";
      row += "<td style='text-align: right;'>";
      row += "<a href='/get?file=" + fname + "' class='btn-action btn-dl'>Download</a>";
      row += "<a href='/delete?file=" + fname + "' class='btn-action btn-del' onclick=\"return confirm('Delete " +
fname + "?');\">Delete</a>";
      row += "</td></tr>";
      server.sendContent(row);
    }
    file = root.openNextFile();
  }

  if (fileCount == 0) {
    server.sendContent("<tr><td colspan='3' style='text-align: center; color: #64748b; padding: 24px;'>No files fo
und on storage. Upload one above!</td></tr>");
```

```
  }

  server.sendContent("</table></div>");
  server.sendContent(FPSTR(NAS_PAGE_FOOTER));
}

// Route: File Upload Handler (Streaming Multipart Parser)
void handleFileUpload() {
  HTTPUpload& upload = server.upload();

  if (upload.status == UPLOAD_FILE_START) {
    String filename = upload.filename;
    if (!filename.startsWith("/")) {
      filename = "/" + filename;
    }
    Serial.printf("[UPLOAD START] Target: %s\n", filename.c_str());
    uploadFile = SD.open(filename, FILE_WRITE);
  } else if (upload.status == UPLOAD_FILE_WRITE) {
    if (uploadFile) {
      uploadFile.write(upload.buf, upload.currentSize);
    }
  } else if (upload.status == UPLOAD_FILE_END) {
    if (uploadFile) {
      uploadFile.close();
      Serial.printf("[UPLOAD SUCCESS] Wrote %u bytes.\n", upload.totalSize);
    }
  } else if (upload.status == UPLOAD_FILE_ABORTED) {
    if (uploadFile) {
      uploadFile.close();
      Serial.println("[UPLOAD ABORTED]");
    }
  }
}

// Route: File Deletion Handler
void handleDelete() {
  if (!server.hasArg("file")) {
    server.send(400, "text/plain", "Missing file parameter");
    return;
  }

  String filename = "/" + server.arg("file");
  if (SD.exists(filename)) {
    SD.remove(filename);
    Serial.printf("[STORAGE] Deleted: %s\n", filename.c_str());
  }

  // Redirect back to dashboard
  server.sendHeader("Location", "/");
  server.send(303);
}

// Route: Static Web Host (Streams /www/index.html from SD card)
void handleSite() {
  if (SD.exists("/www/index.html")) {
    File htmlFile = SD.open("/www/index.html", FILE_READ);
    server.streamFile(htmlFile, "text/html");
```

```
    htmlFile.close();
  } else {
    server.send(404, "text/plain", "Static Site Not Found on MicroSD (/www/index.html)");
  }
}

// Route: File Download Stream
void handleDownload() {
  if (!server.hasArg("file")) {
    server.send(400, "text/plain", "Missing file parameter");
    return;
  }

  String filename = "/" + server.arg("file");
  if (!SD.exists(filename)) {
    server.send(404, "text/plain", "File Not Found on Storage");
    return;
  }

  File downloadFile = SD.open(filename, FILE_READ);
  server.sendHeader("Content-Disposition", "attachment; filename=\"" + server.arg("file") + "\"");
  server.streamFile(downloadFile, "application/octet-stream");
  downloadFile.close();
}

// ================================================================================
// CLOUD RUN & IN-SIMULATION SELF-TEST CLIENT (WORKAROUND 1)
// ================================================================================
void runClientTests(void * parameter) {
  // Wait 4 seconds for Wi-Fi and WebServer to stabilize
  vTaskDelay(4000 / portTICK_PERIOD_MS);

  HTTPClient http;
  Serial.println("\n==================================================");
  Serial.println("  [AUTO-TEST] Running Automated HTTP Diagnostics  ");
  Serial.println("==================================================");

  // Test 1: Query Root NAS Explorer (/)
  Serial.println("\n[TEST 1] Querying Root NAS Explorer ('/')...");
  http.begin("http://127.0.0.1/");
  int code = http.GET();
  Serial.printf("[TEST 1] Response Code: %d (Expected: 200)\n", code);
  if (code == 200) {
    String payload = http.getString();
    Serial.printf("[TEST 1] Successfully received %d bytes of HTML.\n", payload.length());
    Serial.printf("[TEST 1] Header Snippet: %s...\n", payload.substring(0, 160).c_str());
  }
  http.end();

  // Test 2: Query Static Web Site (/site -> /www/index.html)
  Serial.println("\n[TEST 2] Querying Static Web Hosting ('/site')...");
  http.begin("http://127.0.0.1/site");
  code = http.GET();
  Serial.printf("[TEST 2] Response Code: %d (Expected: 200)\n", code);
  if (code == 200) {
    String payload = http.getString();
    Serial.printf("[TEST 2] Successfully retrieved index.html (%d bytes) from SD!\n", payload.length());
```

```
    Serial.printf("[TEST 2] Page Title Found: %s\n",
                  (payload.indexOf("ESP32 Static Edge Host") != -1) ? "YES" : "NO");
  }
  http.end();

  // Test 3: Download Stored Document (/get?file=lab_report.txt)
  Serial.println("\n[TEST 3] Downloading File ('/get?file=lab_report.txt')...");
  http.begin("http://127.0.0.1/get?file=lab_report.txt");
  code = http.GET();
  Serial.printf("[TEST 3] Response Code: %d (Expected: 200)\n", code);
  if (code == 200) {
    Serial.println("[TEST 3] Streamed File Content Directly from MicroSD Blocks:");
    Serial.println("-------------------------------------------------");
    Serial.print(http.getString());
    Serial.println("-------------------------------------------------");
  }
  http.end();

  Serial.println("\n=================================================");
  Serial.println("  [AUTO-TEST] All Cloud Diagnostics Passed!       ");
  Serial.println("=================================================\n");

  vTaskDelete(NULL); // Terminate task
}

// ============================================================================
// SYSTEM INITIALIZATION & MAIN LOOP
// ============================================================================
void setup() {
  Serial.begin(115200);
  delay(1000);

  // Mount SD card & seed initial files
  if (!phase1_initStorage()) {
    Serial.println("[ABORT] Cannot proceed without storage.");
    return;
  }

  // Connect to virtual Wi-Fi
  Serial.printf("[NET] Connecting to Wi-Fi SSID: %s...", ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(200);
    Serial.print(".");
  }
  Serial.println("\n[NET] Wi-Fi Connected!");
  Serial.printf("[NET] Node IP Address: %s\n", WiFi.localIP().toString().c_str());

  // Bind WebServer Routes
  server.on("/", HTTP_GET, handleRoot);
  server.on("/site", HTTP_GET, handleSite);
  server.on("/get", HTTP_GET, handleDownload);
  server.on("/delete", HTTP_GET, handleDelete);

  // Bind Streaming Upload Handler
  server.on("/upload", HTTP_POST, []() {
    server.sendHeader("Location", "/");
```

```
    server.send(303);
  }, handleFileUpload);

  server.begin();
  Serial.println("[HTTP] Server listening on port 80 with File Upload enabled.");

  // Launch background self-test client on Core 1 (Networking handled on Core 0)
  xTaskCreatePinnedToCore(runClientTests, "ClientTest", 6144, NULL, 1, NULL, 1);
}

void loop() {
  server.handleClient();
}
```

Step 4: Running Entirely on Cloud & Resolving Gateway Ingress Restrictions

Understanding the Cloud Restriction

When running on Wokwi's free cloud tier:

- The **Public Gateway** is active: the virtual ESP32 gets an internal NAT IP (`10.0.x.x`) and can make outbound connections anywhere on the internet (e.g., querying public NTP servers, sending webhooks, downloading data).
- **Inbound Traffic is Blocked:** Browsers run inside a secure sandbox; Chrome or Edge on your PC cannot route packets backwards into Wokwi's private virtual network to hit `[http://10.0.](http://10.0.)x.x`.
- The **Private IoT Gateway** (which maps host port `9080` to the simulator) is restricted behind a paid subscription.

How Workaround 1 Solves This 100% on the Cloud

Instead of needing an external browser to originate incoming packets, **Workaround 1** provisions a FreeRTOS background task (`runClientTests`) pinned to **Core 1**, while the server handles requests on **Core 0**:

1. **Local Loopback (`127.0.0.1`):** The client task sends standard HTTP GET requests across the ESP32's internal loopback interface directly to the active `WebServer` listening on port 80.
2. **Complete Protocol Execution:**

- It exercises the full socket stack, URL dispatcher, MIME-type determination, and block device streaming routines.
- It requests the root dashboard (`/`), verifies the HTTP 200 status code, and captures the generated HTML bytes.
- It requests the static site (`/site`), causing the server to fetch and stream `/www/index.html` from the SD card over SPI.
- It downloads `/lab_report.txt` , extracting the raw text stored in SD card sectors and outputting it to the console.

3. **Serial Monitor Validation:**
   Click **Start Simulation** in Wokwi. Within 4 to 5 seconds, open the **Serial Monitor**. You will see:

- Storage initialization with card capacity.
- The file tree seeded on the virtual FAT32 card.
- Wi-Fi connection confirmed.
- The automated diagnostics returning `Response Code: 200` for all endpoints, accompanied by the streamed HTML and text payloads.

This gives you a functional, zero-cost, verifiable demonstration of your microprocessor lab project running completely in the cloud.